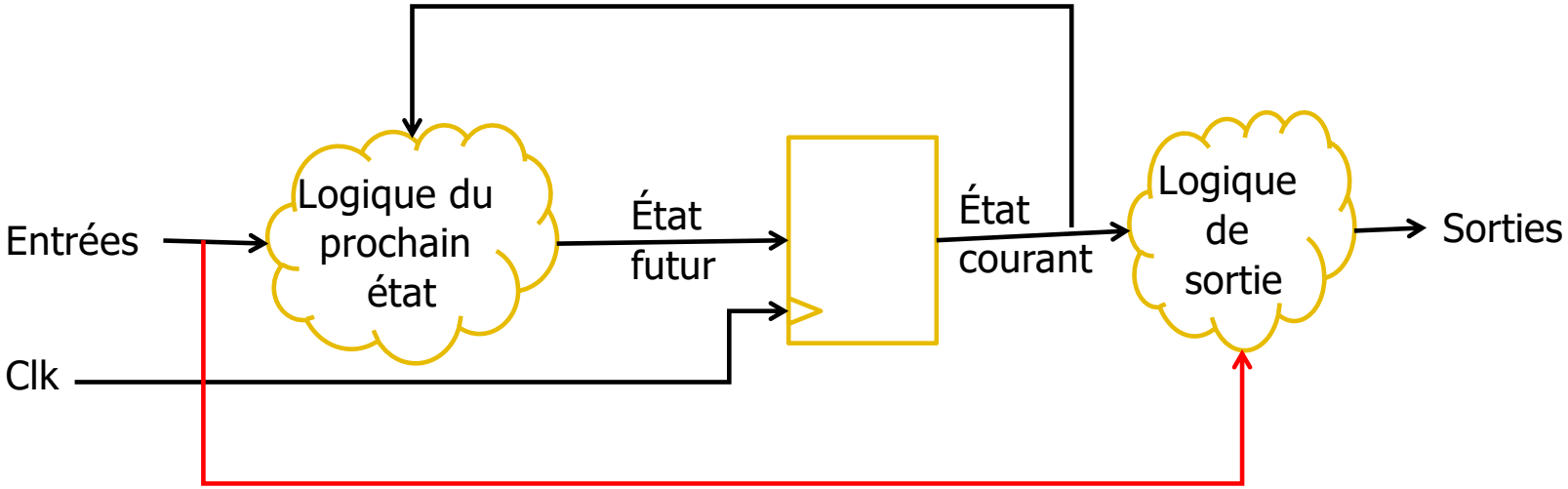
C10 Machine à état (MAE) / Finite State Machine (FSM)

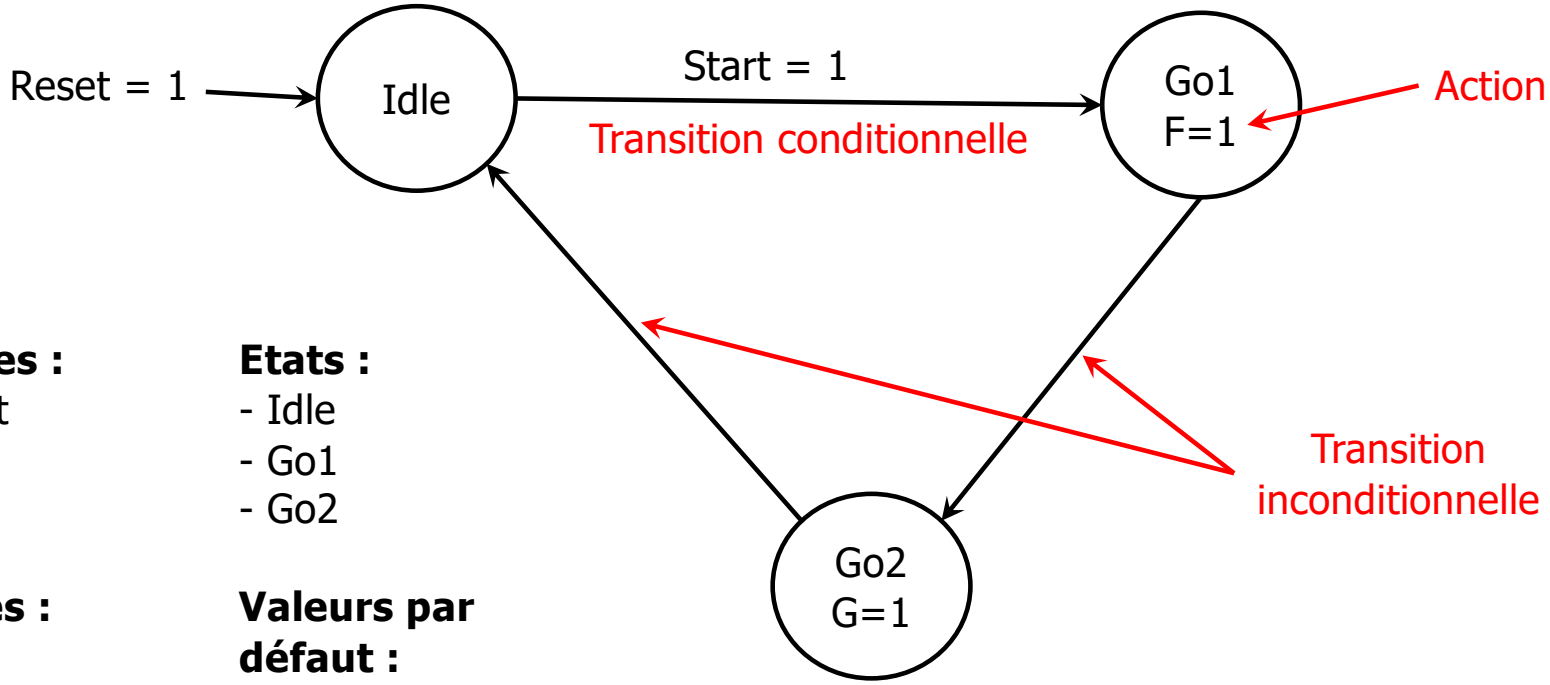
Yann DOUZE

Machine de Moore et Mealy



- Machine de Moore→ les sorties ne dépendent que de l'état courant.
- Machine de Mealy → les sorties dépendent de l'état courant et des entrées.

Diagramme d'état



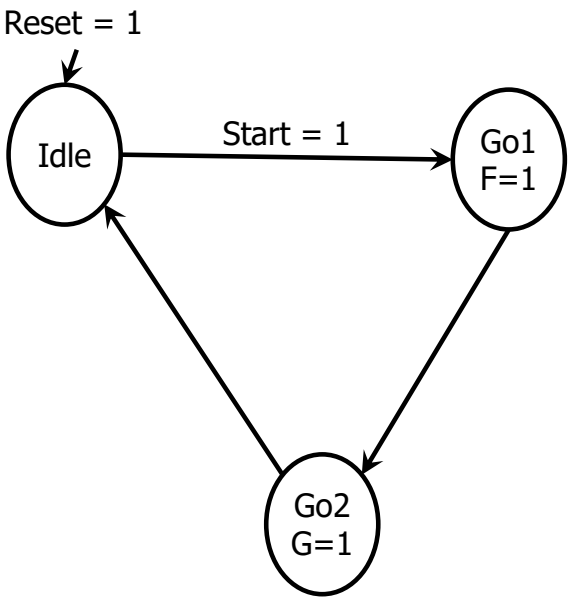
Entrées :
- Reset
- Start
- Clk

Sorties :
- F
- G

Etats :
- Idle
- Go1
- Go2

Valeurs par défaut :
- F = 0
- G = 0

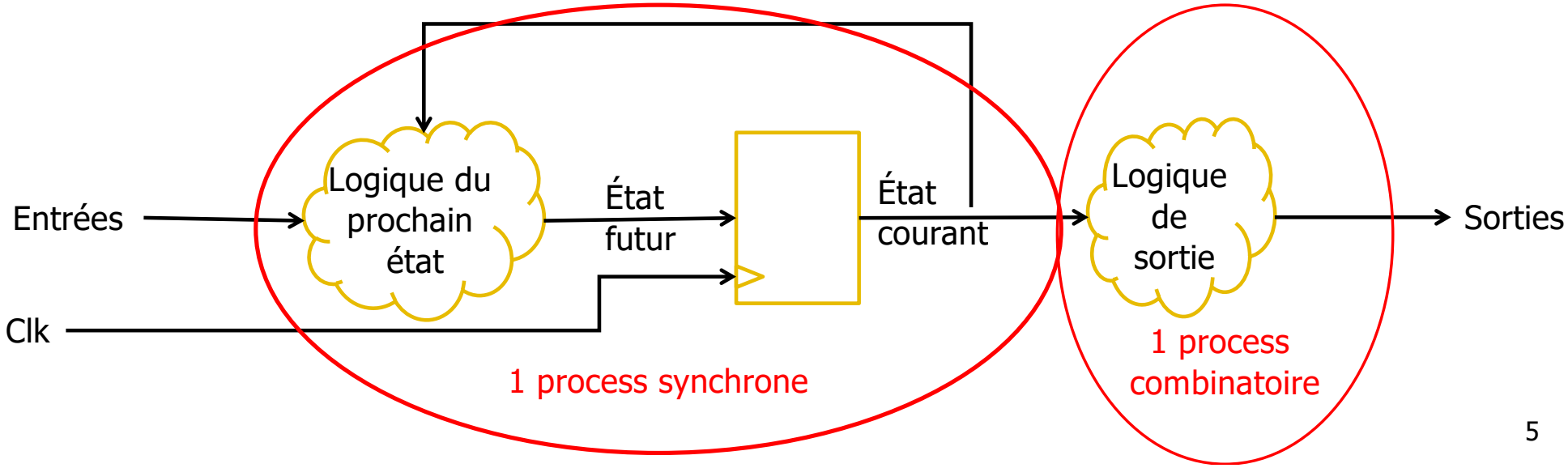
Entité de ce diagramme d'état



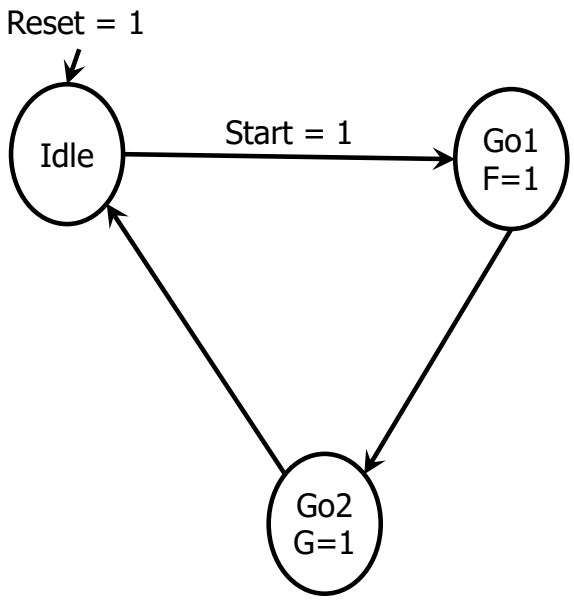

```
entity FSM is
Port (
  Clk,        : in STD_LOGIC;
  Reset       : in STD_LOGIC;
  Start       : in STD_LOGIC;
  F           : out STD_LOGIC;
  G           : out STD_LOGIC
);
end entity;
```

Machine de Moore
(à éviter, moins performante)

- 2 process
  - 1 process synchrone pour déterminer le prochain état en fonction de l'état courant et des entrées.
  - 1 process combinatoire qui permet de déterminer les sorties en fonction de l'état courant.



Description VHDL : Machine de Moore (à éviter, moins performante)

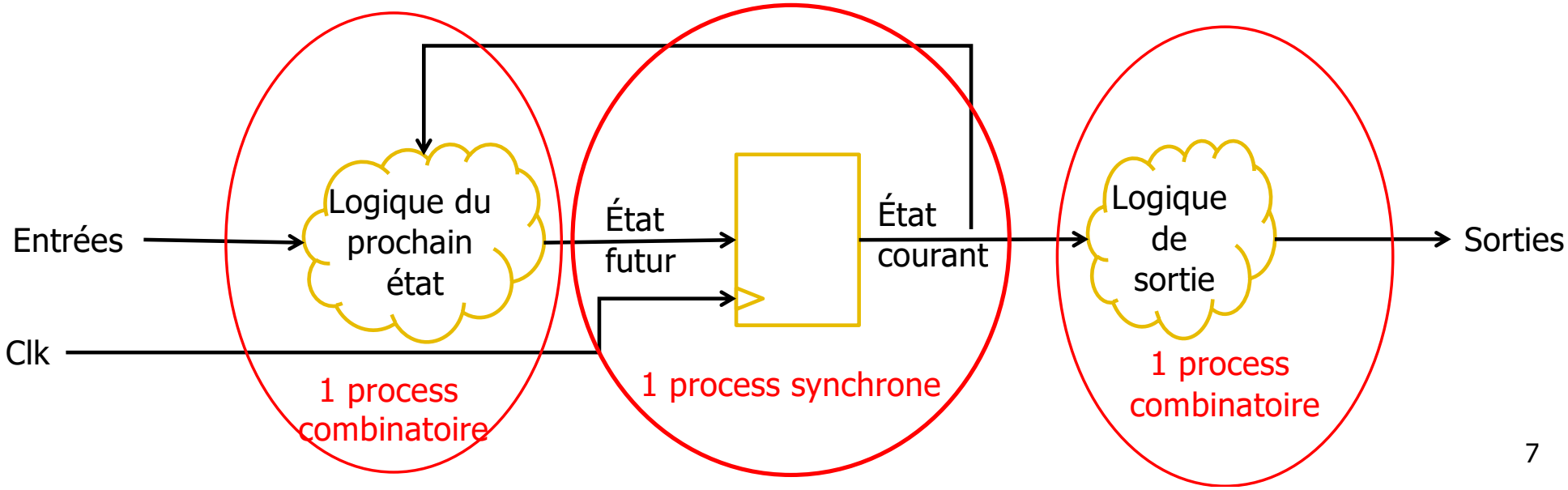



```
architecture Moore of FSM is
  type StateType is (Idle, Go1, Go2);
  Signal State : StateType;
begin
 process(Clk, Reset)
 begin
  if Reset = '1' then
     State <= Idle;
  elsif Rising_edge(Clk) then
    case State is
     when Idle =>
       if Start = '1' then
         State <= Go1;
       end if;
     when Go1 =>
       State <= Go2;
     when Go2 =>
       State <= Idle;
    end case;
  end if;
end process;
```

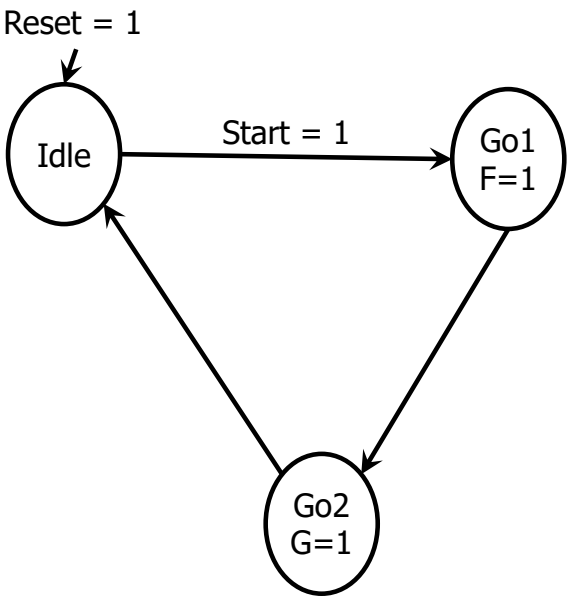
```
process(State)
 begin
   F <= '0';
   G <= '0';
   if State = Go1 then
     F <= '1';
   elsif State = Go2 then
     G <= '1';
   end if;
end process;
end architecture;
```

Machine de Mealy non resynchronisé (à éviter, moins performante)

- 3 process
  - 1 process combinatoire qui permet de déterminer l'état futur en fonction de l'état courant et des entrées.
  - 1 process synchrone pour mémoriser l'état futur dans l'état courant.
  - 1 process combinatoire qui permet de déterminer les sorties en fonction de l'état courant.



Description VHDL : Machine de Mealy non resynchronisé (à éviter, moins performante)

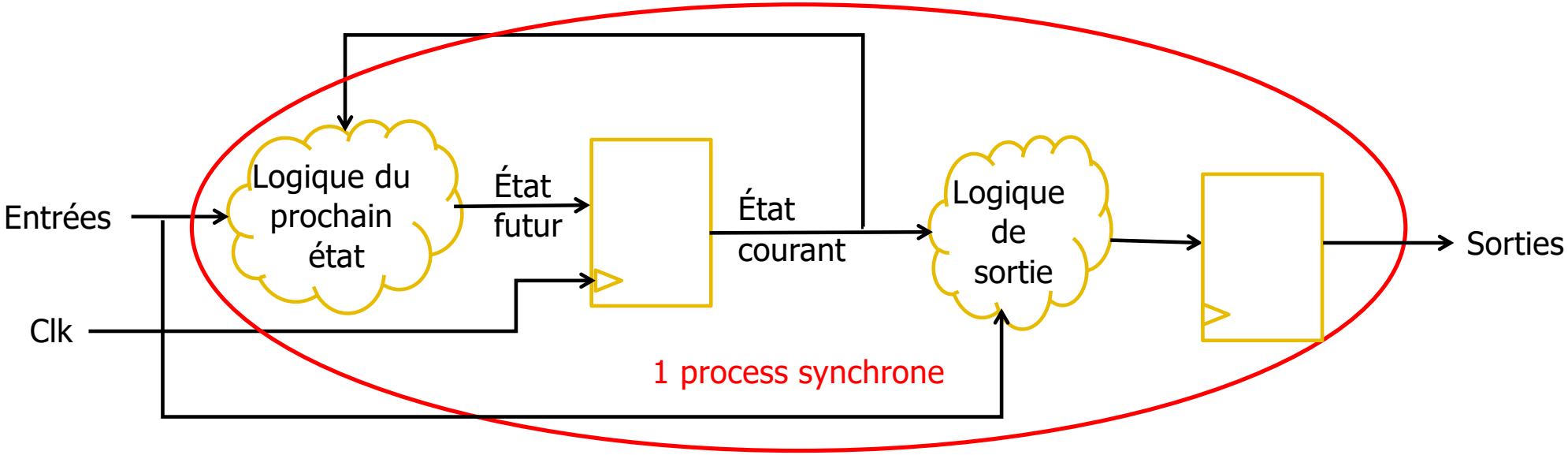



```
architecture Behavioral of EXEMPLE is
  type StateType is (Idle, Go1, Go2);
  signal State, NextState : StateType;
begin

process(State, Start)
begin
 case State is
  when Idle =>
    if Start = '1' then
      NextState <= Go1;
    end if;
  when Go1 =>
    NextState <= Go2;
  when Go2 =>
    NextState <= Idle;
  when others =>
    NextState <= Idle;
 end case;
end process;
```

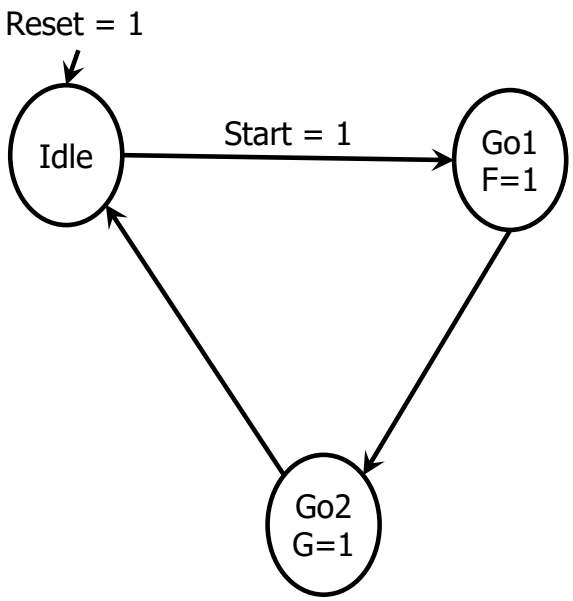
```
process(Clk, Reset)
begin
 if Reset = '1' then
   State <= Idle;
 elsif rising_edge(Clk) then
   State <= NextState;
 end if;
end process;

process(State, Start)
begin
 F <= '0'; G <= '0’;
 case State is
   when Idle =>
     if Start = '1' then
       F <= '1’;
     end if;
   when Go1 =>
     F <= '1’;
   when Go2 =>
     G <= '1’;
 end case;
end process;
end architecture;
```

Machine de Mealy resynchronisé. (recommandé)

- 1 seul process synchrone qui permet en même temps de :
  - Déterminer la logique du prochain état en fonction de l'état courant
  - Déterminer l'état des sorties en fonction de l'état courant et des entrées

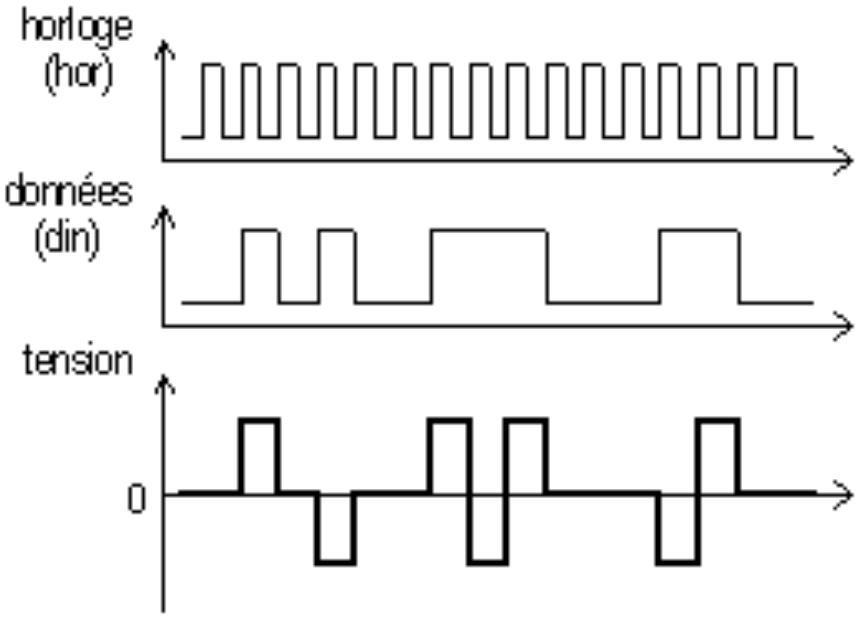


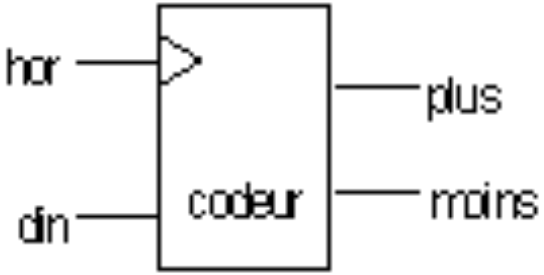
Description VHDL : Machine de Mealy resynchronisé (recommandé)




```
architecture Mealy_resynchro of FSM is
  type StateType is (Idle, Go1, Go2);
  Signal State : StateType;
begin
 process(Clk, Reset)
 begin
   if Reset = ‘1’ then
     State <= Idle;
     G <= ‘0’;
     F <= ‘0’;
   elsif Rising_edge(Clk) then
     case State is
     when Idle =>
       if Start = ‘1’ then
         State <= Go1;
         F <= ‘1’;
       end if;
     when Go1 =>
       State <= Go2;
       G <= '1’;
       F <= '0’;
     when Go2 =>
       State <= Idle;
       G <= '0’;
    end case;
   end if;
end process;
end architecture;
```

Codeur AMI (Alternate Mark Inversion)





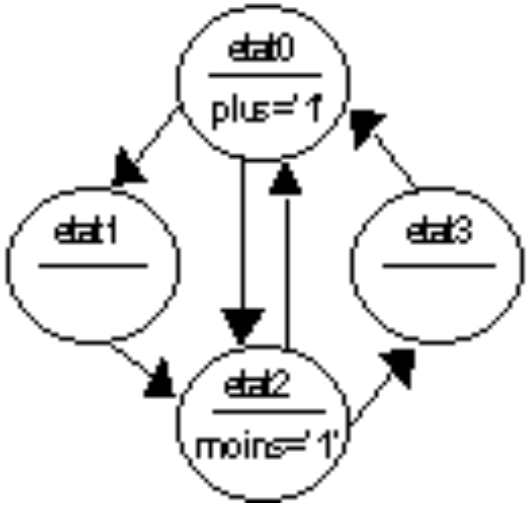
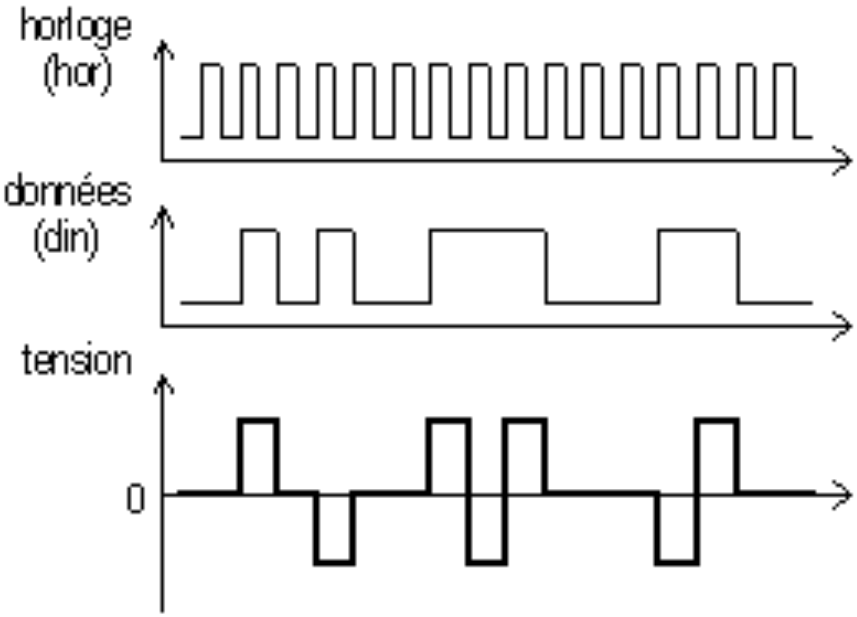
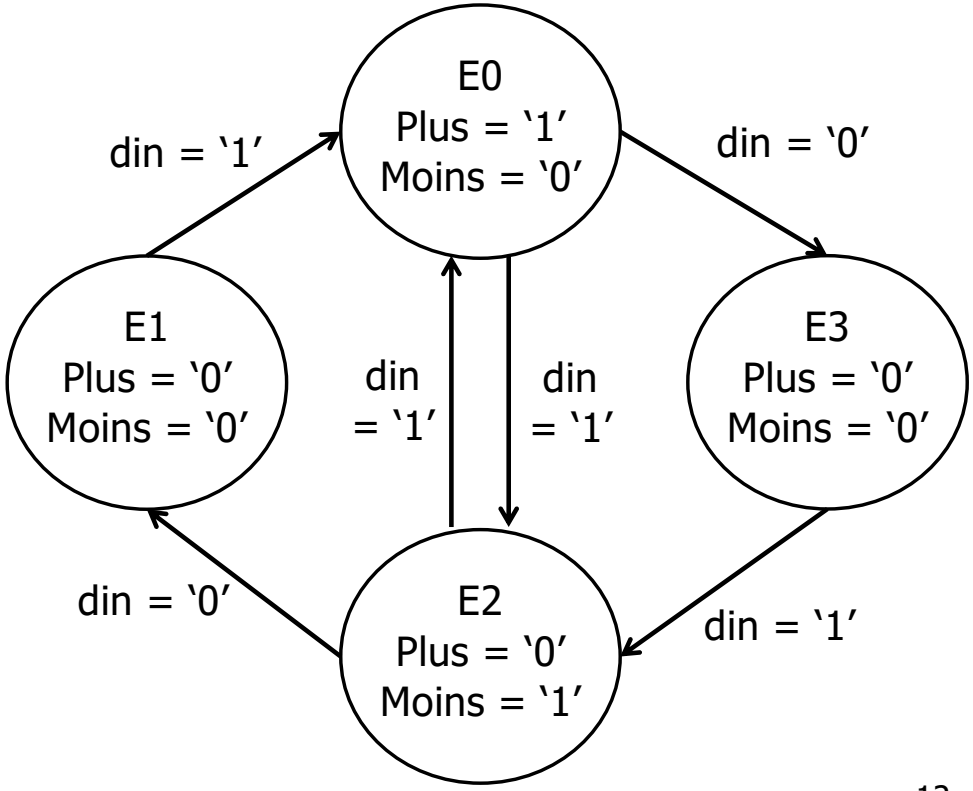
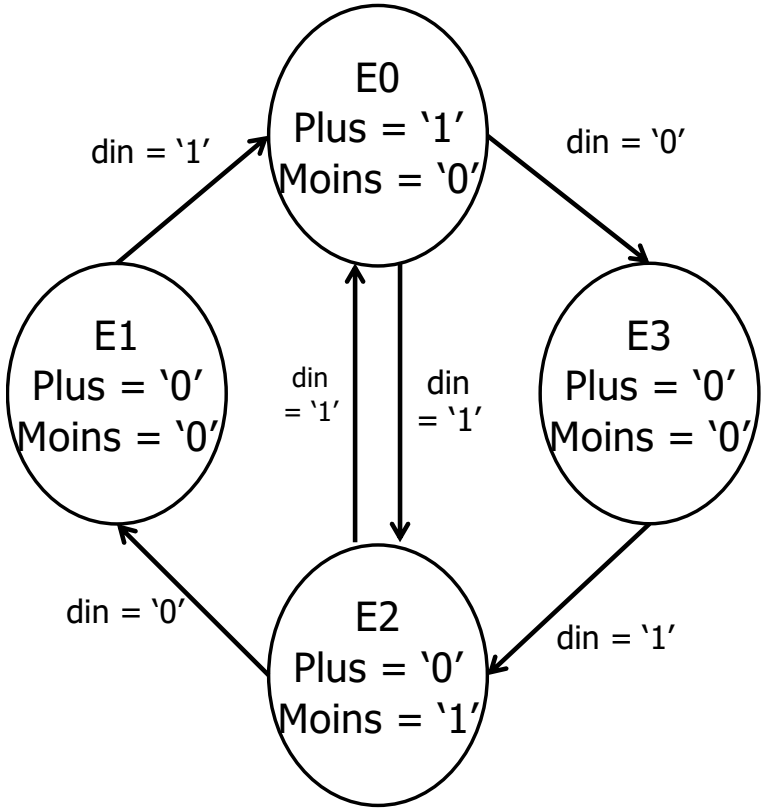


Diagramme d'état du codeur AMI





Code VHDL du codeur AMI




```
architecture MAE of AMI is
   type StateType is (E0, E1, E2, E3);
   Signal State : StateType;
begin
process(Clk, Reset)
begin
   if Reset = '1' then
      State <= E1;
      plus <= '0';
      moins <= '0';
   elsif Rising_edge(Clk) then
      case State is
      when E0 =>if Din = '0' then
                   State <= E3;
                   plus <= '0';
                   moins <= '0';
                elsif Din = '1' then
                   State <= E2;
                   moins <= '1';
                   plus <= '0';
                end if;
      when E1 =>if Din = '1' then
                   State <= E0;
                   moins <= '0';
                   plus <= '1';
                end if;
      when E2 =>if Din = '1' then
                   State <= E0;
                   plus <= '1';
                   moins <= '0';
                elsif Din = '0' then
                   State <= E1;
                   plus <= '0';
                   moins <= '0';
                end if;
      when E3 =>if Din = '1' then
                   State <= E2;
                   plus <= '0';
                   moins <= '1';
                end if;
      end case;
   end if;
end process;
end architecture;
```